Bethlehem University
جامعة بيت لحم

Faculty of Applied Sciences,
Technology and Engineering

Department of
Technology

Software construction

Final Project Report

Project Title

Jayyak — Ride-Hailing & Public Transport Management System

Team Name: Jayyak Dev Team

Sadil Ihab Mahmoud Issa — *202404564* — Database Designer and Technical Writer
Liya Mohammad Al Shami — *202404582* — Creative Designer and Requirements Analyst
Israa Hussain Eid — *202404971* — System Architect and Requirements Engineer

Course: Software Construction
Instructor: *Dr. Suhail Odeh*
Semester: Second Semester / 2026
Submission Date: *25 / 05 / 2026*



Table of Contents

Table of Contents 2
1. Project Overview 3
1.1 Project Purpose & Scope 3
1.2 System Context 3
2. System Requirements 4
2.1 Functional Requirements (FR) 4
2.2 Non-Functional Requirements (NFR) 5
3. MoSCoW Prioritization 7
4. Data Flow Diagrams (DFD) 9
4.1 Context Diagram 9
External Entities & Data Flows 9
4.2 Level 0 – Detailed DFD 10
Process Descriptions 10
5. UML Diagrams 12
5.1 Use Case Diagram 12
Use Case Descriptions ..... **Error! Bookmark not defined.**
5.2 Class Diagram 13
Key Classes Summary 14
5.3 State Chart Diagram 16
State Descriptions 17
5.4 Sequence Diagrams 18
Sequence Diagram — Scenario 1: Normal Ride Request & Payment 18
Sequence Diagram — Scenario 2: Emergency Dispatch & Payment Failure ..... **Error! Bookmark not defined.**
Message Flow Summary 19
6. Entity-Relationship Diagram (ERD) 22
6.1 ERD Diagram 22
6.2 Entity Descriptions 22
6.3 Relationship Table 24
7. Database Normalization 25
7.1 Unnormalized Form (UNF) 25
7.2 First Normal Form (1NF) 25
8. References & Appendices 29
8.1 References 29
8.2 Appendices 29
Appendix A – Glossary of Terms 29
Appendix B – Team Contribution Table 30

1. Project Overview

1.1 Project Purpose & Scope

Jayyak is a mobile and web-based ride-hailing and public transport management system designed for the Palestinian market. It addresses the fragmented and inefficient transport landscape by providing a structured platform for passengers to book rides, drivers to receive fair and organized assignments through a rotation queue, and institutions to manage student transport subscriptions. The system delivers transparent pricing, safe ride preferences, and integrated digital payment — all in one platform.

In Scope	Out of Scope
• Passenger and driver registration & profile management • Ride request with preference selection (gender, urgency) • Advance (pre-booked) ride scheduling with deposit • Driver rotation queue management • Emergency/urgent dispatch bypassing the queue • Checkpoint status display for drivers • QR code payment via in-app wallet (bank integrated) • Post-trip driver rating system • Subscription management for students	• Inter-city or international route management • Vehicle maintenance tracking • Insurance and legal compliance management • Physical cash payment handling • Ride-sharing between multiple unrelated passengers • Third-party advertising within the platform

1.2 System Context

Jayyak operates as a centralized platform connecting passengers, drivers, and financial institutions. The system manages ride lifecycles from request to payment and supports both individual and institutional (student subscription) users. External payment processors are integrated to handle wallet transactions securely.

Stakeholder	Role / Interest	Interaction with System
Passenger	Primary user — books and pays for rides	Uses the mobile app to request, track, and pay for trips
Driver	Service provider — accepts and completes ride requests	Receives assignments via the app, navigates, collects payment via QR
System Administrator	Oversees platform operations, manages users and reports	Accesses the admin dashboard for monitoring and control
Palestine Islamic Bank / Arab Bank	External payment processor	Processes wallet top-up and ride payment transactions
University / Institution	Operator — manages student transport subscriptions	Coordinates bulk subscriptions and route assignments

2. System Requirements

2.1 Functional Requirements (FR)

Functional requirements describe what the system must do — the specific behaviors, features, and functions it must provide. Each requirement is uniquely identified, prioritized, and paired with measurable acceptance criteria.

The functional and non-functional requirements below were identified through two structured **gathering methods**: a field interview conducted with Ahmad Zaoul, a public transport driver with 23 years of experience on the Husan–Bethlehem route, and a public survey completed by 141 participants from the Bethlehem University community and surrounding areas. Full documentation of both methods — including the interview transcript, template, and survey results — is available at: Jayyak – Interview and Survey Documentation.

ID	Requirement Name	Description	Priority	Acceptance Criteria
FR-01	Passenger Registration	The system shall allow passengers to register with full name, phone number, home location, and destination address. A unique ID is assigned upon registration.	High	Passenger can log in and view their profile after registering. All fields are validated.
FR-02	Driver Registration	The system shall allow drivers to register with name, years of experience, vehicle details (type, model, plate), and a scanned driving license. Account is pending admin approval.	High	Driver profile is stored; admin receives notification; driver cannot accept rides until approved.
FR-03	Ride Request with Preferences	Passengers shall be able to submit a ride request selecting a ride preference: Women-Only, Men-Only, Mixed, or Urgent.	High	Request is recorded with preference; system filters eligible drivers accordingly.
FR-04	Pre-Booked Ride	Passengers shall be able to schedule a ride in advance by specifying date/time and paying a refundable deposit. The system sends a reminder 30 minutes before pickup.	High	Booking is confirmed after deposit; reminder is sent; if cancelled late, partial deduction is applied.
FR-05	Driver Rotation Queue	The system shall maintain a FIFO rotation queue per route. The next eligible driver in queue is automatically assigned to each new request.	High	Assignments follow queue order; driver moves to end of queue after completing a trip.
FR-06	Emergency Dispatch	Urgent requests (e.g., medical, student emergency) shall bypass the queue and dispatch the nearest available driver immediately.	High	Driver receives emergency notification; pickup is initiated within 5 minutes.
FR-07	Real-Time Trip Tracking	During an active trip, the passenger shall be able to view the driver's location and ETA on a live map.	Medium	GPS coordinates update every 10 seconds; map reflects current position.

FR-08	**Checkpoint Status Display**	**Drivers shall see a real-time status (open/closed) for checkpoints on their route to enable dynamic rerouting.**	**Medium**	**Checkpoint status is accurate within 5 minutes; driver receives alternate route suggestion when closed.**
FR-09	**QR Code Payment**	**After trip completion, the passenger scans the driver's QR code in-app. The fare is displayed and deducted from the passenger's wallet upon confirmation.**	**High**	**Payment is processed within 3 seconds; both parties receive a confirmation notification.**
FR-10	**Bank Wallet Integration**	**The system shall integrate with Palestine Islamic Bank and Arab Bank APIs to allow passengers to top up their in-app wallet.**	**High**	**Top-up is reflected in wallet balance within 10 seconds of bank confirmation.**
FR-11	**Post-Trip Rating**	**Passengers shall rate the driver after payment, scoring the vehicle (cleanliness, comfort) and the driver (behavior, punctuality), and optionally leaving a comment.**	**Medium**	**Rating is stored and averaged into driver profile score after each submission.**
FR-12	**Subscription Management**	**Institutions (e.g., universities) shall be able to register students on a recurring transport subscription tied to a specific route.**	**Medium**	**Subscribed students are automatically enrolled for daily trips on their route without repeated requests.**
FR-13	**Driver Arrival Alert**	**The system shall automatically notify the passenger when the assigned driver is within a defined proximity (≤ 500 meters) of the pickup location.**	**High**	**Passenger receives push notification when driver is ≤ 500 meters away Notification includes driver name, vehicle plate, and ETA Alert is triggered automatically without any manual action from the driver**

2.2 Non-Functional Requirements (NFR)

Non-functional requirements define the quality attributes of the system — how it performs, scales, and behaves rather than what it does.

ID	Category	Description	Metric / Benchmark	Rationale
NFR-01	**Performance**	**The system shall respond to ride requests and payment actions within 2 seconds under normal load (up to 500 concurrent users).**	**≤ 2 s for 95% of requests**	**Ensures a smooth, frustration-free experience for time-sensitive transport bookings.**

NFR-02	**Security**	**All user passwords shall be hashed using bcrypt with a minimum cost factor of 12. All API communication shall use HTTPS/TLS 1.2+.**	**Industry standard (OWASP)**	**Protects sensitive personal and financial data from unauthorized access.**
NFR-03	**Scalability**	**The system architecture shall support horizontal scaling to handle up to 5,000 concurrent users without degradation.**	**Load tested at 5,000 users**	**As Jayyak expands to new cities, the system must grow without re-architecting.**
NFR-04	**Usability**	**The mobile interface shall achieve a minimum SUS (System Usability Scale) score of 75 in user testing.**	**SUS ≥ 75**	**Ensures accessibility for users with varying technical literacy levels.**
NFR-05	**Availability**	**The system shall maintain an uptime of ≥ 99.5% measured monthly, excluding scheduled maintenance windows.**	**99.5% uptime**	**Transport is time-critical; unplanned downtime directly impacts passengers and drivers.**
NFR-06	**Data Privacy**	**The system shall comply with applicable data protection regulations. Passenger location data shall not be retained beyond 30 days after trip completion.**	**Data retention ≤ 30 days**	**Builds user trust and limits liability from excessive personal data storage.**
NFR-07	**Localization**	**The system shall support both Arabic and English languages, with Arabic as the default language for all user-facing interfaces.**	**Full UI available in Arabic and English**	**The primary user base is Arabic-speaking; bilingual support ensures wider accessibility.**
NFR-08	**Compatibility**	**The mobile application shall be compatible with Android 8.0+ and iOS 13.0+ devices.**	**Supports Android 8.0+ and iOS 13.0+**	**Covers the majority of active devices in the Palestinian market.**

3. MoSCoW Prioritization

The MoSCoW method categorizes requirements by priority, helping the team focus on what matters most for the project's success and guiding development sprint planning.

MUST Have	SHOULD Have	COULD Have	WON'T Have
Critical — cannot launch without	*Important — include if possible*	*Nice-to-have — if time allows*	*Excluded — out of scope this release*

ID	Feature / Requirement	Justification	Linked FR/NFR	Priority
F-01	**Passenger Registration**	**Without registered users, no trips can be booked. This is the entry point of the entire system.**	**FR-01**	**Must**
F-02	**Driver Registration & Approval**	**Drivers must be verified before accepting passengers to ensure safety and accountability.**	**FR-02**	**Must**
F-03	**Ride Request with Preferences**	**Core feature — matching passengers to appropriate drivers is the primary value proposition.**	**FR-03**	**Must**
F-04	**Driver Rotation Queue**	**Ensures fair driver income distribution. Central to Jayyak's differentiation from other platforms.**	**FR-05**	**Must**
F-05	**QR Code Payment & Wallet**	**Cashless payment is essential for a professional, trackable transport service.**	**FR-09, FR-10**	**Must**
F-06	**Emergency Dispatch**	**Critical for student safety and time-sensitive cases. Non-negotiable for institutional clients.**	**FR-06**	**Must**
F-07	**Compatibility**	**App must support Android 8.0+ and iOS 13.0+ to cover the majority of devices in the Palestinian market.**	**NFR-08**	**Mist**
	Localization	**Full Arabic and English UI support. Arabic as default — primary user base is Arabic-speaking.**	**NFR-07**	**Should**
F-07	**Pre-Booked Ride Scheduling**	**Significantly improves user convenience; institutions require predictable scheduling.**	**FR-04**	**Should**
F-08	**Real-Time GPS Tracking**	**Improves passenger confidence and safety during trips.**	**FR-07**	**Should**

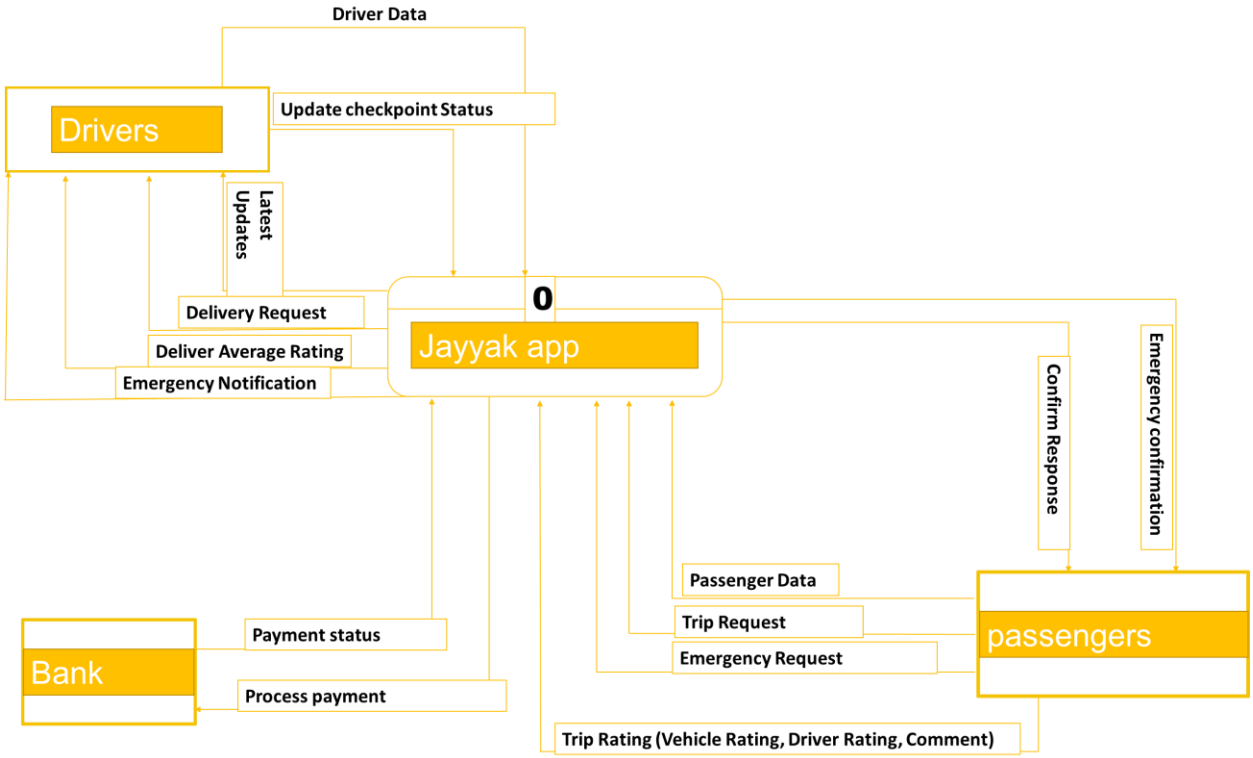
F-09	**Post-Trip Rating System**	**Provides quality control and driver accountability feedback loop.**	**FR-11**	**Should**
F-10	**Checkpoint Status Display**	**Helps drivers navigate efficiently; reduces delays caused by closed checkpoints.**	**FR-08**	**Should**
F-11	**Driver Arrival Alert**	**Notifies passenger when driver is nearby. Significantly reduces no-shows and improves experience.**	**FR-13**	**Should**
F-12	**Subscription Management**	**Valuable for institutional contracts but not required at initial launch.**	**FR-12**	**Could**
F-13	**In-App Chat (Driver-Passenger)**	**Convenient but calls or existing messaging apps suffice for MVP.**	**—**	**Could**
F-14	**Driver Performance Analytics Dashboard**	**Useful for admins but not essential for core operations at launch.**	**NFR-04**	**Could**
F-15	**Physical Cash Payment**	**App-first model; cash introduces reconciliation complexity. Planned for a later phase.**	**—**	**Won't**
F-16	**Inter-City Route Management**	**Scope is local/institutional routes. Long-distance routing requires separate regulatory compliance.**	**—**	**Won't**
F-17	**Third-Party Advertising**	**Conflicts with user experience principles and trust. Explicitly excluded.**	**—**	**Won't**

4. Data Flow Diagrams (DFD)

Data Flow Diagrams illustrate how data moves through the Jayyak system. The Context Diagram presents the system as a single process with its external entities; Level 0 decomposes the system into its major internal processes.

4.1 Context Diagram

The Level 0 diagram shows Jayyak as a single process. External entities send data to and receive data from the system, representing the system boundary.

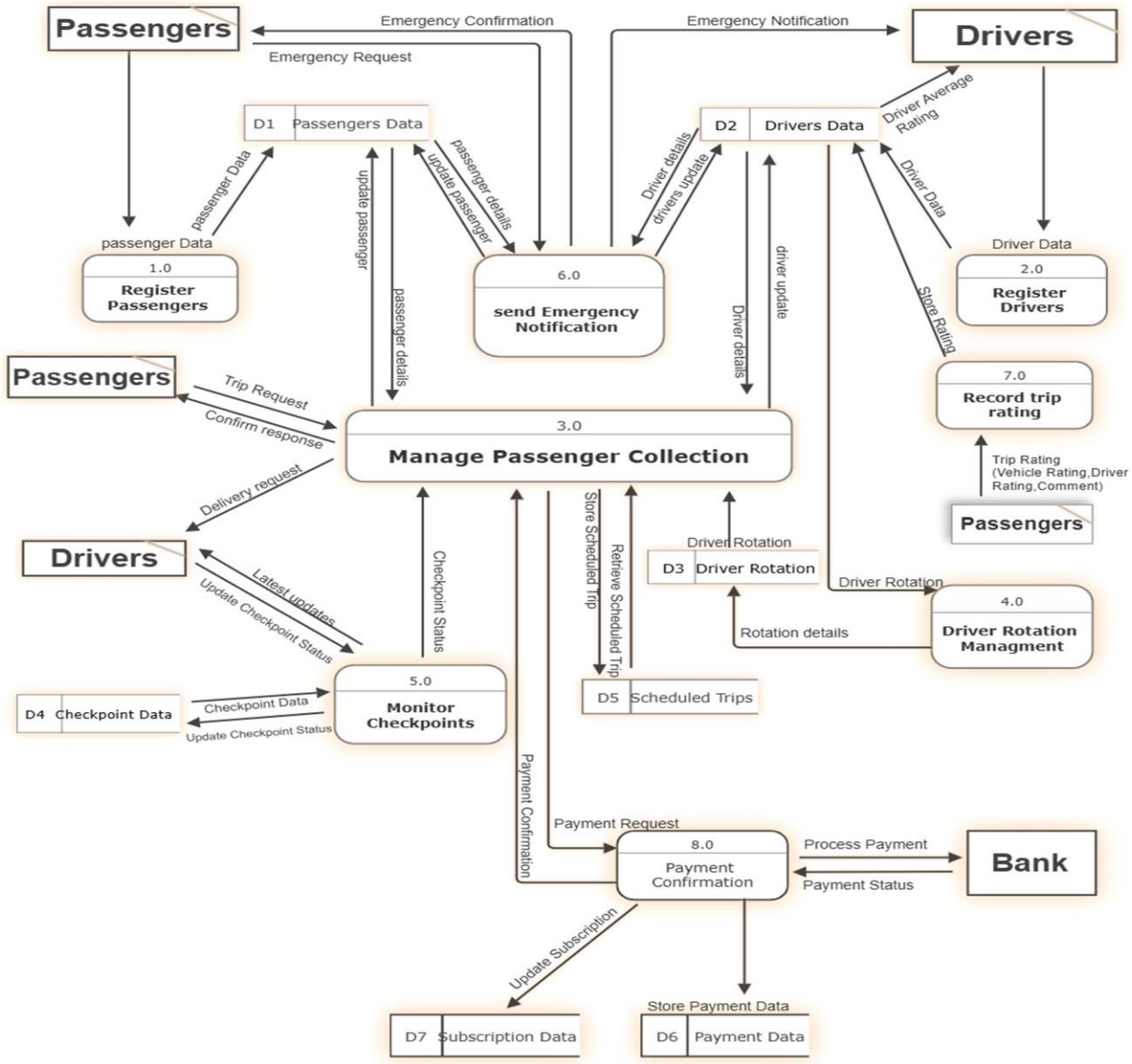


External Entities & Data Flows

External Entity	Data Flowing In	Data Flowing Out
Passenger	**Passenger Data, Trip Request, Emergency Request, Trip Rating (Vehicle Rating, Driver Rating, Comment)**	**Confirm Response, Emergency Confirmation**
Driver	**Driver Data, Update Checkpoint Status**	**Delivery Request, Deliver Average Rating, Emergency Notification, Latest Updates**
Bank (PIB / Arab Bank)	**Payment Status**	**Process Payment**

4.2 Level 0 – Detailed DFD

The Level 0 diagram decomposes the Jayyak system into seven major processes, showing data stores, sub-process interactions, and the data flowing between them.



Process Descriptions

Process ID	Process Name	Description	Data Stores Used
P1 (1.0)	Register Passengers	Handles new passenger registration and stores profile data	D1
P2 (2.0)	Register Drivers	Handles driver registration, stores driver data and rating	D2

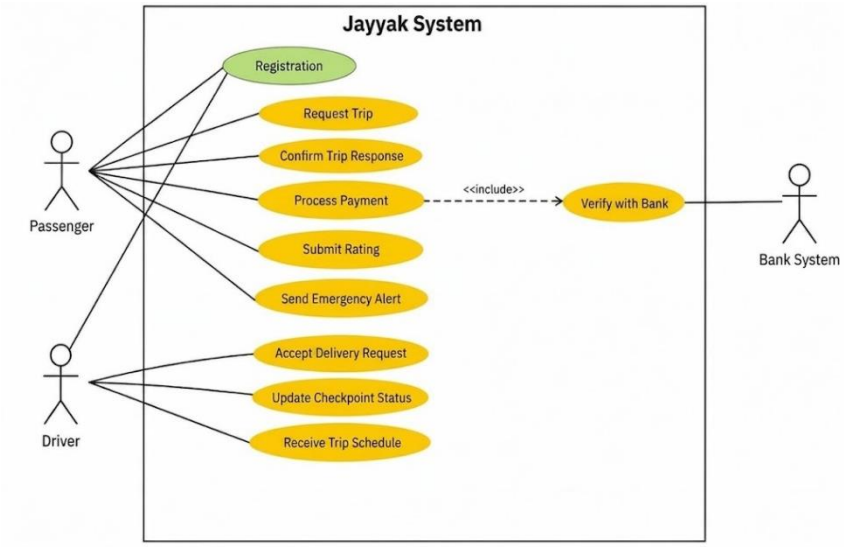
P3 (3.0)	**Manage Passenger Collection**	**Core process — handles trip requests, confirms responses, coordinates driver delivery requests and latest updates**	**D1, D2, D3, D5**
P4 (4.0)	**Driver Rotation Management**	**Manages the driver rotation queue, retrieves and updates rotation details**	**D3**
P5 (5.0)	**Monitor Checkpoints**	**Reads checkpoint data, updates checkpoint status, sends status to main process**	**D4**
P6 (6.0)	**Send Emergency Notification**	**Sends emergency notifications to drivers and confirmation to passengers, updates passenger and driver data**	**D1, D2**
P7 (7.0)	**Record Trip Rating**	**Receives trip rating from passenger (vehicle, driver, comment) and stores it to driver data**	**D2**

5. UML Diagrams

UML (Unified Modeling Language) diagrams provide standardized visual representations of the Jayyak system's structure and behavior. The diagrams below cover use cases, class structure, the trip state lifecycle, and interaction sequences for critical scenarios.

5.1 Use Case Diagram

The use case diagram illustrates the interactions between the system's actors — Passenger, Driver, Admin, and Institution — and the functional capabilities of Jayyak. Key relationships include <<include>> for mandatory sub-flows and <<extend>> for optional extensions.

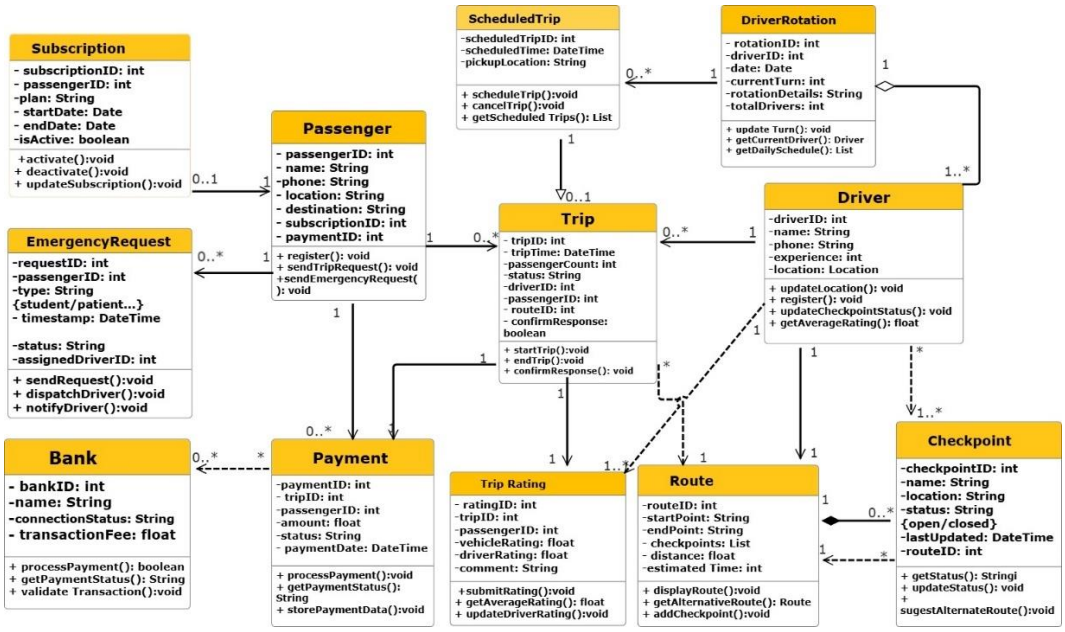


UC ID	Use Case Name	Actor(s)	Precondition	Basic Flow Summary
UC-01	Registration	Passenger / Driver	App installed; user not registered	User opens app → selects role (Passenger or Driver) → fills registration form → submits → account created
UC-02	Request Trip	Passenger	Passenger registered and logged in	Passenger enters origin and destination → selects ride preference → submits trip request → awaits confirmation
UC-03	Confirm Trip Response	Passenger	Trip request submitted	System assigns driver → passenger receives confirmation response with driver details.
UC-04	Process Payment	Passenger	Trip completed	Passenger scans driver QR code → fare displayed → confirms payment → system includes Verify with Bank <<include>>.

UC-05	**Submit Rating**	**Passenger**	**Payment confirmed**	**Passenger receives rating screen → scores vehicle and driver (1–5) → adds optional comment → submits.**
UC-06	**Send Emergency Alert**	**Passenger**	**Passenger logged in**	**Passenger triggers emergency → system sends alert → nearest driver notified → confirmation sent back to passenger.**
UC-07	**Accept Delivery Request**	**Driver**	**Driver registered, logged in, in queue**	**Driver receives trip assignment notification → reviews passenger details → accepts request.**
UC-08	**Update Checkpoint Status**	**Driver**	**Driver logged in and on active route**	**Driver reports checkpoint as open or closed → system updates checkpoint data for all active drivers.**
UC-09	**Receive Trip Schedule**	**Driver**	**Driver registered and active**	**Driver views upcoming scheduled trips assigned by the rotation queue.**
UC-10	**Verify with Bank**	**Passenger / Bank System**	**Payment initiated by passenger**	**System sends payment request to bank → Bank verifies and processes transaction → returns payment status to system.**

5.2 Class Diagram

The class diagram shows the static structure of the Jayyak system — its core domain classes, their attributes and methods, and the relationships between them. The User class serves as a base for the Passenger and Driver roles via inheritance.



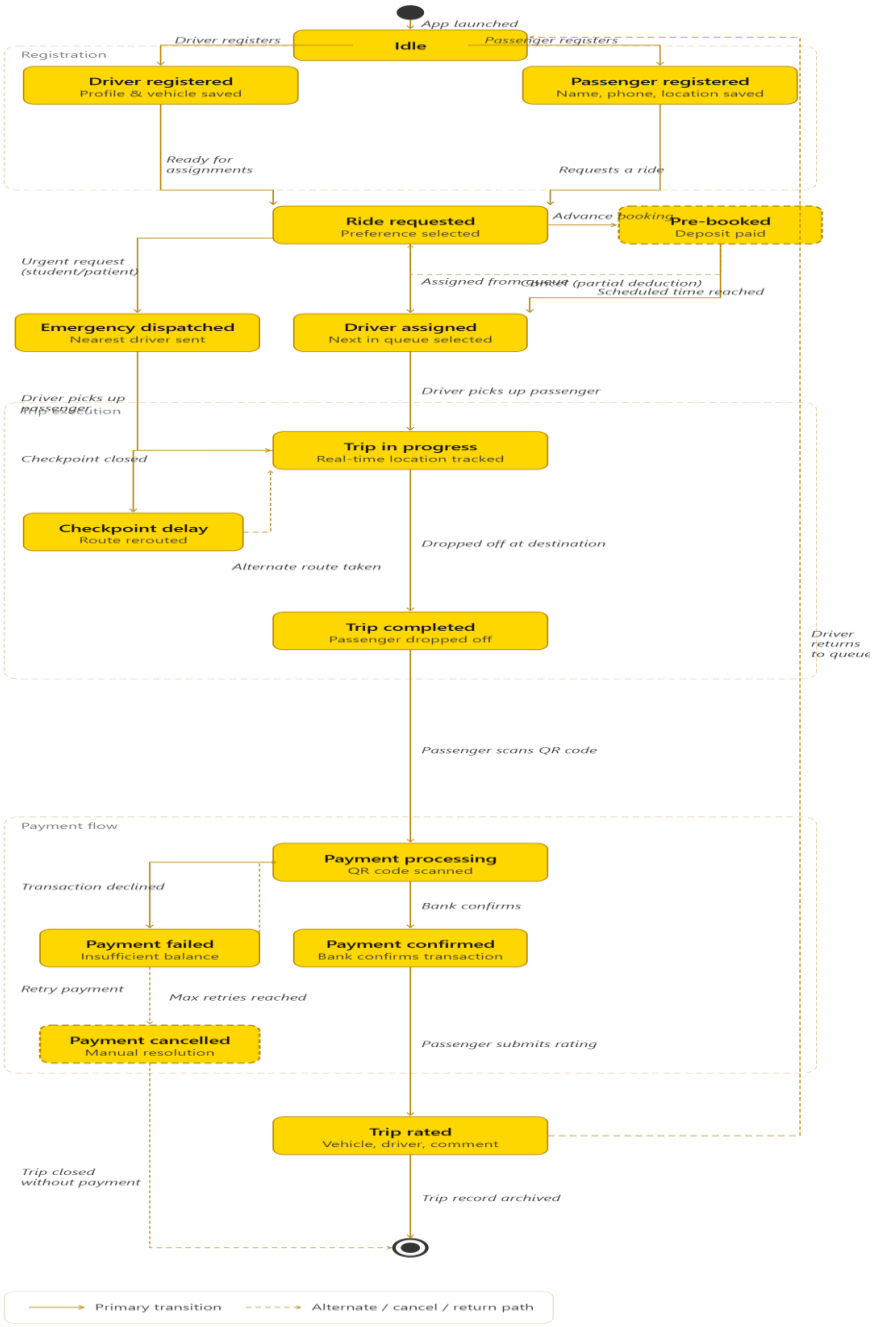
Key Classes Summary

Class Name	Key Attributes	Key Methods	Relationships
Passenger	passengerID: int, name: String, phone: String, location: String, destination: String, subscriptionID: int, paymentID: int	register(), sendTripRequest(), sendEmergencyRequest()	1 to 0..* Trip; 0..1 to 1 Subscription; 1 to 0..* EmergencyRequest
Driver	driverID: int, name: String, phone: String, experience: int, location: Location	updateLocation(), register(), updateCheckpointStatus(), getAverageRating()	1 to 0..* Trip; 1..* to 1 DriverRotation; 1 to 1..* Checkpoint (dependency)
Trip	tripID: int, tripTime: DateTime, passengerCount: int, status: String, driverID: int, passengerID: int, routeID: int, confirmResponse: boolean	startTrip(), endTrip(), confirmResponse()	0..* to 1 Passenger; 0..* to 1 Driver; 0..1 to 1 ScheduledTrip; 1 to 1..* TripRating; * to 1 Route
Payment	paymentID: int, tripID: int, passengerID: int, amount: float, status: String, paymentDate: DateTime	processPayment(), getPaymentStatus(), storePaymentData()	0..* to 1 Passenger; 0..* to * Bank (dependency)
TripRating	ratingID: int, tripID: int, passengerID: int, vehicleRating: float, driverRating: float, comment: String	submitRating(), getAverageRating(), updateDriverRating()	1..* to 1 Trip; linked to Driver via updateDriverRating()
Bank	bankID: int, name: String, connectionStatus: String, transactionFee: float	processPayment(), getPaymentStatus(), validateTransaction()	* to 0..* Payment (dependency)
ScheduledTrip	scheduledTripID: int, scheduledTime: DateTime, pickupLocation: String	scheduleTrip(), cancelTrip(), getScheduledTrips()	0..* to 1 DriverRotation; 0..1 to 1 Trip
DriverRotation	rotationID: int, driverID: int, date: Date, currentTurn: int, rotationDetails:	updateTurn(), getCurrentDriver(), getDailySchedule()	1 to 1..* Driver; 1 to 0..* ScheduledTrip (aggregation)

	String, totalDrivers: int		
EmergencyRequest	**requestID: int, passengerID: int, type: String, timestamp: DateTime, status: String, assignedDriverID: int**	**sendRequest(), dispatchDriver(), notifyDriver()**	**0..* to 1 Passenger**
Route	**routeID: int, startPoint: String, endPoint: String, checkpoints: List, distance: float, estimatedTime: int**	**displayRoute(), getAlternativeRoute(), addCheckpoint()**	*** to 1..* Trip (dependency); 1 to 0..* Checkpoint**
Checkpoint	**checkpointID: int, name: String, location: String, status: String, lastUpdated: DateTime, routeID: int**	**getStatus(), updateStatus(), suggestAlternateRoute()**	**0..* to 1 Route; 1..* to 1 Driver (dependency)**
Subscription	**subscriptionID: int, passengerID: int, plan: String, startDate: Date, endDate: Date, isActive: boolean**	**activate(), deactivate(), updateSubscription()**	**0..1 to 1 Passenger**

5.3 State Chart Diagram

The state chart diagram models the complete lifecycle of a Trip entity — the central object in the Jayyak system. It covers every state from the initial system launch through ride request, execution, payment, and final archiving.



State Descriptions

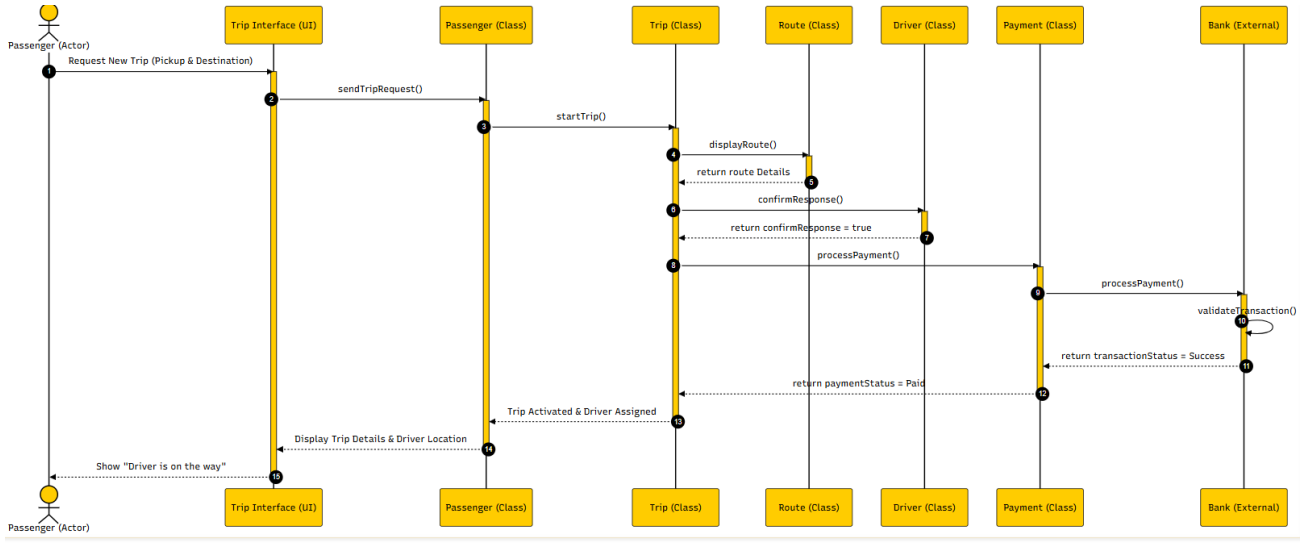
State	Description	Entry Action	Exit Condition / Transition
Idle	System is running, no active trip. Drivers are in the rotation queue waiting for assignments.	Initialize system; place drivers in queue	Driver registers → Driver Registered; Passenger registers → Passenger Registered
Driver Registered	Driver profile, vehicle details, and license information are saved and pending approval.	Store driver profile and vehicle data	Driver approved and ready → Ride Requested (receives assignments)
Passenger Registered	Passenger profile created with name, phone, and location saved.	Store passenger data	Passenger requests a ride → Ride Requested
Ride Requested	A trip request is active with passenger preference selected (gender, urgency). System evaluates assignment logic.	Log request with timestamp and preference	Assigned from queue → Driver Assigned; Urgent flag → Emergency Dispatched; Advance booking selected → Pre-Booked
Pre-Booked	Ride is scheduled in advance with deposit deducted from wallet. System schedules a reminder.	Deduct deposit; schedule reminder notification	Scheduled time reached → Driver Assigned; Passenger cancels → partial deduction applied → Ride Requested
Emergency Dispatched	Urgent request bypasses the rotation queue. Nearest available driver is alerted immediately.	Send emergency alert to nearest driver	Driver picks up passenger → Trip In Progress
Driver Assigned	Next eligible driver selected from the rotation queue and notified of the trip details.	Notify driver; lock driver to this trip	Driver picks up passenger → Trip In Progress
Trip In Progress	Active trip. Passenger is in the vehicle. Real-time GPS tracking is live.	Start GPS stream; begin fare calculation	Checkpoint closed → Checkpoint Delay; Passenger dropped off → Trip Completed
Checkpoint Delay	A checkpoint on the route is closed. Driver is rerouting via an alternate path.	Send reroute alert to driver	Alternate route confirmed → Trip In Progress
Trip Completed	Passenger has been dropped off at destination. GPS tracking stops and final fare is calculated.	Stop GPS stream; calculate final fare	Passenger scans QR code → Payment Processing
Payment Processing	QR code has been scanned. Fare is displayed and system awaits passenger confirmation and bank response.	Display fare; send payment request to bank	Bank confirms → Payment Confirmed; Transaction declined → Payment Failed

Payment Failed	**Transaction was declined by the bank or wallet balance is insufficient.**	**Notify passenger of failure; increment retry counter**	**Passenger retries → Payment Processing; Max retries reached → Payment Cancelled**
Payment Cancelled	**Payment could not be completed after maximum retries. Trip is closed without successful payment for manual resolution.**	**Set trip status to closed; notify admin**	**Trip closed → Final State (archived)**
Payment Confirmed	**Bank has authorized and processed the transaction successfully. Receipt sent to both parties.**	**Update wallet balance; log transaction; send receipt**	**Passenger submits rating → Trip Rated**
Trip Rated	**Passenger submits vehicle rating, driver rating (1–5), and an optional comment. Driver's average score is updated.**	**Display rating screen**	**Trip record archived → Final State; Driver returns to rotation queue → Idle**

5.4 Sequence Diagrams

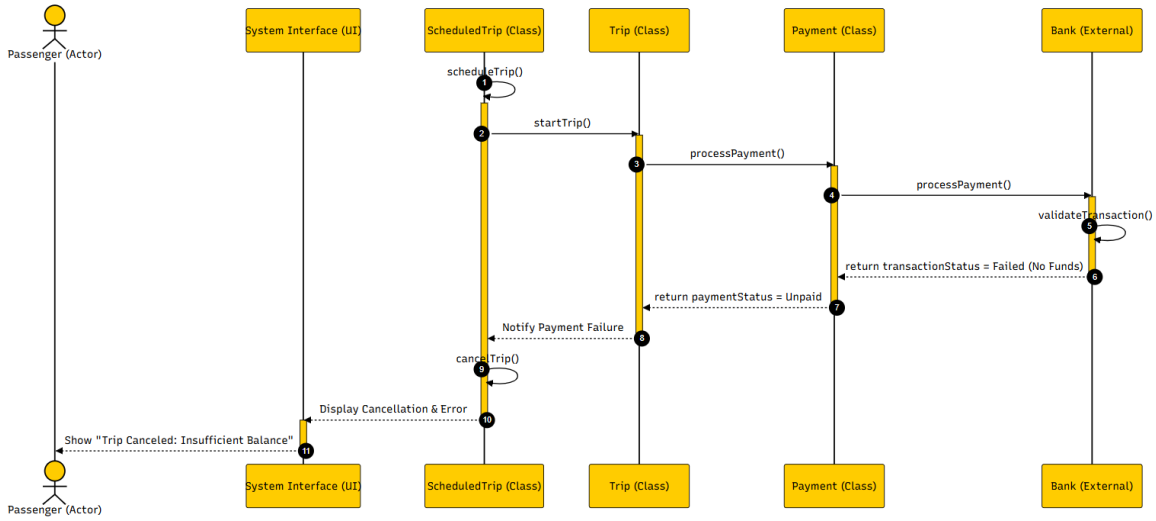
Sequence Diagram — Scenario 1: Successful Ride Request & Payment

This scenario models the primary "Happy Path" of the application. It illustrates the end-to-end workflow where a registered passenger requests a new ride with specific preferences. The system processes the request, assigns the next available driver from the rotation queue, computes the route, executes successful payment processing via the external Bank API layer, and activates the trip.



Sequence Diagram — Scenario 2: Failed Payment & Scheduled Trip Cancellation

This scenario models an alternative error-handling execution path within the system. It demonstrates a pre-scheduled trip reaching its designated activation time, where the system auto-initializes the workflow but the transaction is rejected due to insufficient funds during the external banking check. The system preserves integrity by executing an automatic cancellation policy and notifying the user.



Message Flow Summary

(Scenario 1)

Step	Sender	Receiver	Message / Method	Purpose
1	**Passenger (Actor)**	**Trip Interface (UI)**	**Request New Trip (Pickup & Destination)**	**The passenger inputs pickup and destination details to request a ride.**
2	**Trip Interface (UI)**	**Passenger (Class)**	`sendTripRequest()`	**The interface triggers the request method inside the Passenger class.**
3	**Passenger (Class)**	**Trip (Class)**	`startTrip()`	**The Passenger object instantiates a concrete Trip object to begin processing.**
4	**Trip (Class)**	**Route (Class)**	`displayRoute()`	**The Trip class invokes the Route object to calculate distance and estimated time.**

5	**Route (Class)**	**Trip (Class)**	`return route Details`	**The Route class returns the calculated coordinates and path details back to Trip.**
6	**Trip (Class)**	**Driver (Class)**	`confirmResponse()`	**The Trip class updates the rotation logic to assign and ping an eligible driver.**
7	**Driver (Class)**	**Trip (Class)**	`return confirmResponse = true`	**The Driver acknowledges and confirms acceptance of the assigned ride request.**
8	**Trip (Class)**	**Payment (Class)**	`processPayment()`	**Once confirmed, the Trip object triggers payment processing to capture the fare.**
9	**Payment (Class)**	**Bank (External)**	`processPayment()`	**The Payment component forwards the financial criteria to the external Bank API.**
10	**Bank (External)**	**Bank (External)**	`validateTransaction()`	**The Bank system executes an internal self-validation check for balance and security.**
11	**Bank (External)**	**Payment (Class)**	`return transactionStatus = Success`	**The Bank returns a successful response confirming captured funds.**
12	**Payment (Class)**	**Trip (Class)**	`return paymentStatus = Paid`	**The Payment layer notifies the Trip object that billing was settled successfully.**
13	**Trip (Class)**	**Passenger (Class)**	`Trip Activated & Driver Assigned`	**The Trip status shifts to active, linking the assigned driver details to the Passenger.**
14	**Passenger (Class)**	**Trip Interface (UI)**	`Display Trip Details & Driver Location`	**The Passenger profile pushes the updated trip parameters and live data to the UI layer.**
15	**Trip Interface (UI)**	**Passenger (Actor)**	**Show "Driver is on the way"**	**The UI displays a live notification to the passenger tracking the oncoming vehicle.**

Message Flow Summary

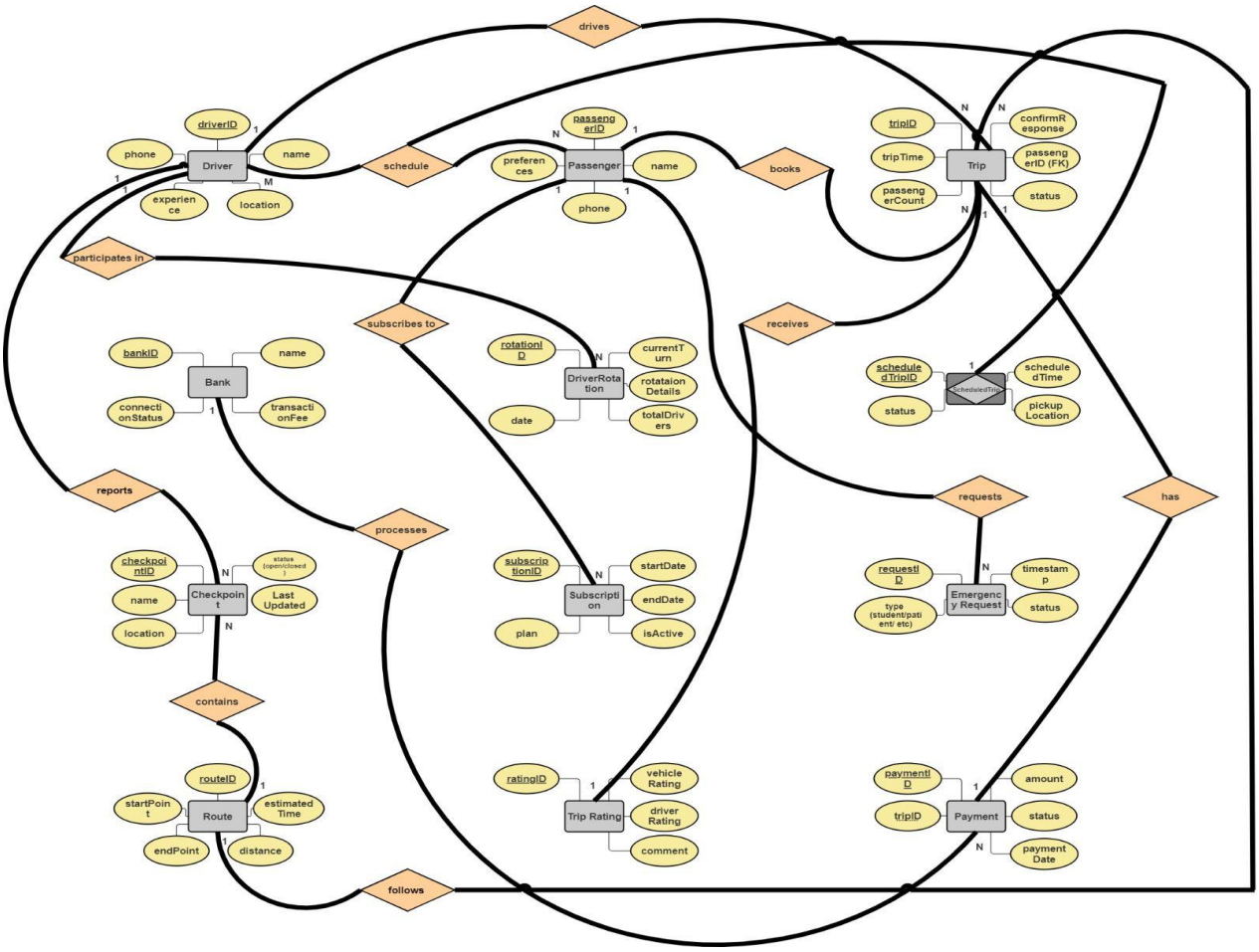
(Scenario 2)

Step	Sender	Receiver	Message / Method	Purpose
1	**ScheduledTrip (Class)**	**ScheduledTrip (Class)**	`scheduleTrip()`	**The system automated cron triggers the pre-booked trip's active verification window.**
2	**ScheduledTrip (Class)**	**Trip (Class)**	`startTrip()`	**The ScheduledTrip component calls the Trip entity initialization routines.**
3	**Trip (Class)**	**Payment (Class)**	`processPayment()`	**The Trip object sends a payment validation instruction to ensure transaction clearance.**
4	**Payment (Class)**	**Bank (External)**	`processPayment()`	**The Payment class securely communicates criteria to the integrated Bank interface.**
5	**Bank (External)**	**Bank (External)**	`validateTransaction()`	**The Bank processes an internal balance sweep to audit available funds.**
6	**Bank (External)**	**Payment (Class)**	`return transactionStatus = Failed (No Funds)`	**The Bank gateway returns a failure code indicating an insufficient wallet balance.**
7	**Payment (Class)**	**Trip (Class)**	`return paymentStatus = Unpaid`	**The Payment class reports the transaction rejection status back to the Trip manager.**
8	**Trip (Class)**	**ScheduledTrip (Class)**	`Notify Payment Failure`	**The Trip lifecycle controller updates the scheduling component regarding billing failure.**
9	**ScheduledTrip (Class)**	**ScheduledTrip (Class)**	`cancelTrip()`	**The ScheduledTrip object triggers its self-termination routine to cancel the booking.**
10	**ScheduledTrip (Class)**	**System Interface (UI)**	`Display Cancellation & Error`	**The backend application layer dispatches cancellation logs and error statuses to the UI.**
11	**System Interface (UI)**	**Passenger (Actor)**	**Show "Trip Canceled: Insufficient Balance"**	**The UI alerts the user that their scheduled ride was cancelled due to billing failure.**

6. Entity-Relationship Diagram (ERD)

The ERD describes the Jayyak database structure — its entities, attributes, primary/foreign keys, and the relationships between them. Crow's Foot notation is used throughout for cardinality.

6.1 ERD Diagram



6.2 Entity Descriptions

Entity	Primary Key	Key Attributes	Relationships	Description
Driver	**driverID**	**name, phone, experience, location**	**M:N with Passenger (schedule); 1:1 with Bank (reports); N:M with Checkpoint (participates in)**	**Represents a transport provider registered in the system**
Passenger	**passengerID**	**name, phone, preferences**	**N:1 with Trip (books); N:1 with DriverRotation (subscribes to); N:1 with**	**Represents a user who books and pays for rides**

			EmergencyRequest (requests)	
Trip	tripID	tripTime, passengerCount, status, passengerID (FK), confirmResponse	N:1 with Passenger (books); N:1 with Driver (drives); 1:N with Payment (has); 1:1 with ScheduledTrip (receives); 1:1 with TripRating	Central entity representing a single ride lifecycle
ScheduledTrip	scheduledTripID	scheduledTime, pickupLocation, status	1:1 with Trip (receives)	Represents a pre-booked ride with scheduled time and pickup location
DriverRotation	rotationID	date, currentTurn, rotationDetails, totalDrivers	N:1 with Passenger (subscribes to)	Manages the FIFO driver assignment queue and rotation details
Bank	bankID	name, connectionStatus, transactionFee	1:1 with Driver (reports); 1:N with Payment (processes)	External bank entity that processes payments and reports to drivers
Checkpoint	checkpointID	name, location, status (open/closed), lastUpdated	N:M with Driver (participates in); N:1 with Route (contains)	Represents a road checkpoint with real-time open/closed status
Route	routeID	startPoint, endPoint, distance, estimatedTime	1:N with Checkpoint (contains); follows Trip	Represents a defined transport corridor used during trips
Subscription	subscriptionID	plan, startDate, endDate, isActive	N:1 with Passenger	Represents an institutional or individual transport subscription
EmergencyRequest	requestID	type (student/patient/etc), timestamp, status	N:1 with Passenger (requests); N:1 with Trip	Represents an urgent ride request that bypasses the rotation queue
TripRating	ratingID	vehicleRating, driverRating, comment	1:1 with Trip	Stores post-trip feedback

				submitted by the passenger
Payment	paymentID	tripID, amount, status, paymentDate	N:1 with Trip (has); N:1 with Bank (processes)	Records the financial transaction associated with a completed trip

6.3 Relationship Table

Entity A	Entity B	Cardinality	Relationship Description
Driver	Passenger	M : N	A driver serves many passengers over time; a passenger can be served by many drivers (schedule)
Driver	Checkpoint	N : M	A driver passes through many checkpoints; a checkpoint is used by many drivers (participates in)
Driver	Bank	1 : 1	Each driver is linked to one bank for payment reporting
Passenger	Trip	N : 1	A passenger can have many trips; each trip belongs to one passenger (books)
Passenger	DriverRotation	N : 1	Many passengers are managed through one rotation system (subscribes to).
Passenger	EmergencyRequest	1 : N	A passenger can submit many emergency requests over time (requests)
Trip	Driver	N : 1	Many trips can be driven by one driver over time (drives)
Trip	ScheduledTrip	1 : 1	A pre-booked trip has exactly one scheduled trip record (receives)
Trip	TripRating	1 : 1	Each completed trip receives exactly one rating
Trip	Payment	1 : N	One trip can have one or more payment attempts (has)
Trip	Route	N : 1	Many trips follow one route (follows)
Bank	Payment	1 : N	One bank processes many payment transactions (processes)
Checkpoint	Route	N : 1	Many checkpoints belong to one route (contains)
Subscription	Passenger	N : 1	Many subscriptions can belong to one passenger

7. Database Normalization

Normalization organizes the Jayyak database to eliminate redundancy, prevent update anomalies, and improve data integrity. The following demonstrates the normalization process from the raw unnormalized form through 3NF.

7.1 Unnormalized Form (UNF)

The raw data collected from the system contains multi-valued attributes and potential redundancies. The table below shows the initial flat structure before any normalization, using the Trip class as the primary example of a UNF violation.

Raw Trip table (UNF):

tripID (PK)	tripTime	passengerCount	status	driverID (FK)	passengerID	routeID (FK)	confirmResponse
1	2025-01-10 08:00	3	Completed	1	1, 5, 8	1	true
2	2025-01-11 09:00	2	InProgress	2	2, 14	2	true
3	2025-01-11 12:00	5	Completed	1	15, 4, 7, 8, 2	1	true

Issues identified in UNF:

The passengerID column contains multiple values in a single cell, which violates atomicity — the core requirement of 1NF.

7.2 First Normal Form (1NF)

Eliminate repeating groups; ensure every column contains a single, indivisible value. Each row is uniquely identifiable by a primary key.

Violation Identified	How It Was Resolved
passengerID column in Trip contains multiple values per cell (e.g. "1, 5, 8") — violates atomicity	Created a separate TripPassenger junction table with one row per (tripID, passengerID) pair. Each cell now holds exactly one value.

Passenger table — already in 1NF:

passengerID (PK)	Name	Phone	Location	Destination	subscriptionID	paymentID
1	Sara Hasan	0599000010	Nablus	Ramallah	1	1
2	Lina Naser	0599000011	Jenin	Nablus	2	2

passengerID is the primary key. Each cell contains a single atomic value — 1NF is achieved.

TripPassenger junction table (after 1NF fix on Trip):

tripID (FK)	passengerID (FK)
1	1
1	5
1	8
2	2
2	14
3	15
3	4
3	7

Checkpoint table — already in 1NF:

checkpointID (PK)	Name	Location	Status	lastUpdated	routeID (FK)
1	Hawara	South Nablus	Open	2025-01-10 07:00	1
2	Zatara	North Ramallah	Closed	2025-01-10 08:30	1

7.3 Second Normal Form (2NF)

All tables use single-column primary keys, so partial dependencies (which only occur with composite keys) are not applicable. 2NF is satisfied across all tables. The Route table however violates 2NF due to a partial dependency:

Partial Dependency Found	Dependent Attribute(s)	Resolution (New Table Created)
routeID → estimatedTime, but estimatedTime does not fully depend on routeID alone (varies per trip)	estimatedTime	Decomposed into Route(routeID, startPoint, endPoint, distance) and RouteTime(estimatedTime, routeID FK)

Route table after 2NF:

routeID (PK)	startPoint	endPoint	distance
1	Nablus	Ramallah	60.5
2	Jenin	Nablus	45.0

RouteTime table after 2NF:

estimatedTime (PK)	routeID (FK)
75	1
55	2
70	1

7.4 Third Normal Form (3NF)

Transitive Dependency Found	Dependent Attribute(s)	Resolution
In Passenger: subscriptionID → paymentID (paymentID depends on subscriptionID, not directly on passengerID)	paymentID	Extracted Payment table linked to Subscription via subscriptionID FK. Passenger table retains only subscriptionID FK.
In Checkpoint: name → location (location depends on the checkpoint name, not directly on checkpointID)	location	Split into two tables: CheckpointInfo(checkpointID, name, location) and CheckpointStatus(checkpointID, status, lastUpdated, routeID FK)

Passenger table after 3NF:

passengerID (PK)	Name	Phone	Location	Destination
1	Sara Hasan	0599000010	Nablus	Ramallah
2	Lina Naser	0599000011	Jenin	Nablus

Subscription table after 3NF:

subscriptionID (PK)	passengerID (FK)
1	1
2	2

Payment table after 3NF:

paymentID (PK)	subscriptionID (FK)
1	1
2	2

CheckpointInfo after 3NF:

checkpointID (PK)	Name	Location
1	Hawara	South Nablus
2	Zatara	North Ramallah

CheckpointStatus after 3NF:

checkpointID (PK)	Status	lastUpdated	routeID (FK)
1	Open	2025-01-10 07:00	1
2	Closed	2025-01-10 08:30	1

7.5 Functional Dependencies Summary

Functional Dependency	Table	NF Violated	Action Taken
passengerID → {name, phone, location, destination, subscriptionID, paymentID}	Passenger	3NF	paymentID removed; extracted to Payment table linked via subscriptionID
subscriptionID → paymentID	Passenger	3NF	Created separate Payment(paymentID, subscriptionID FK) table
checkpointID → {name, location, status, lastUpdated, routeID}	Checkpoint	3NF	name → location transitive dependency resolved by splitting into CheckpointInfo and CheckpointStatus tables
routeID → {startPoint, endPoint, distance, estimatedTime}	Route	2NF	estimatedTime separated into RouteTime(estimatedTime, routeID FK) table
tripID → {tripTime, passengerCount, status, driverID, passengerID, routeID, confirmResponse}	Trip	1NF	Multi-valued passengerID extracted into TripPassenger(tripID FK, passengerID FK) junction table

8. References & Appendices

8.1 References

#	Reference (IEEE / APA format)
[1]	Valacich, J. S., & George, J. F. (2024). *Modern Systems Analysis and Design* (10th ed.). Pearson Education.
[2]	Fowler, M. (2003). UML Distilled: A Brief Guide to the Standard Object Modeling Language (3rd ed.). Addison-Wesley Professional.
[3]	Date, C. J. (2003). An Introduction to Database Systems (8th ed.). Addison-Wesley.
[4]	SmartDraw. (2025). Online diagramming tool — used for DFD and ERD creation. https://www.smartdraw.com. Accessed: May 2025.
[5]	Lucidchart. (2025). Online diagram and UML tool — used for use case, class, and sequence diagrams. https://www.lucidchart.com. Accessed: May 2025.
[6]	Palestine Islamic Bank. (2024). Developer API Documentation. https://www.palestineislamicbank.com. Accessed: May 2025.
[7]	Anthropic. (2025). Claude AI Assistant. https://claude.ai. Used for diagram review and documentation structure guidance.

8.2 Appendices

Appendix A – Glossary of Terms

Term	Definition
Jayyak	The Arabic-language ride-hailing and public transport management system developed in this project. The name means 'It came to you' in Palestinian dialect.
Rotation Queue	A First-In-First-Out (FIFO) ordering of available drivers on a given route. Ensures equal distribution of ride assignments among drivers.
Pre-Booked Ride	A ride scheduled in advance by a passenger, requiring a deposit. The system holds the booking until the scheduled time, then assigns a driver automatically.
Emergency Dispatch	A ride request flagged as urgent (e.g., medical emergency or student late for exam) that bypasses the rotation queue and assigns the nearest available driver.
QR Code Payment	A contactless payment method where the passenger scans a unique QR code on the driver's device to trigger a fare deduction from their in-app wallet.
Checkpoint	A physical road checkpoint in the West Bank that may be open or closed. The Jayyak system displays real-time checkpoint status to drivers for route planning.
In-App Wallet	A digital payment balance held within the Jayyak app, topped up via bank integration (Palestine Islamic Bank or Arab Bank).

MoSCoW	A prioritization framework categorizing features into: Must Have, Should Have, Could Have, and Won't Have for a given release.
UNF / 1NF / 2NF / 3NF	Stages of database normalization: Unnormalized Form, First Normal Form, Second Normal Form, Third Normal Form.
DFD	Data Flow Diagram — a graphical representation of data movement through a system, showing processes, data stores, external entities, and data flows.
UML	Unified Modeling Language — a standardized visual language for specifying, constructing, and documenting software systems.
ERD	Entity-Relationship Diagram — a data model diagram showing entities, their attributes, and the relationships between them.
FR	Functional Requirement — a requirement describing a specific behavior or function the system must perform.
NFR	Non-Functional Requirement — a requirement specifying quality attributes such as performance, security, or availability.
SUS	System Usability Scale — a widely used standardized questionnaire (score 0–100) for evaluating perceived usability.

Appendix B – Team Contribution Table

Member Name	Student ID	Sections Contributed	Contribution %
Sadil Ihab Mahmoud Issa	202404564	Regarding Data Modeling and System Behavior, she designed the **Entity-Relationship Diagram (ERD)** for database structuring, developed the **State Machine Diagram (State Diagram)** to model system behavior and transitions, and performed advanced database optimization including Normalization up to 3NF verification for key tables. In Research and Requirements Gathering, she formulated and developed the project survey design. For Documentation and Content Creation, she executed the technical Documentation writing for the project report and contributed to developing and structuring specific sections of the Presentation content.	33%

Israa Hussain Eid	202404971	Regarding Requirements Gathering and Documentation, she performed the full audio transcription and analysis of the project interview, defined and documented the system's Non-Functional Requirements, and applied the MoSCoW Prioritization Technique to categorize and prioritize system requirements. In System Architecture and Data Modeling, she designed the Data Flow Diagrams including both **DFD Level 0 (Context Level Diagram)** and **DFD Level 1 Diagram**, developed the system's **Class Diagram** to represent the structural model, and contributed to the database design phase during the Normalization process.	33%
Liya waleed mohammad Alshami	202404582	Regarding Brand Identity and Conceptualization, she designed the official project logo, selected the system name, and actively brainstormed and conceptualized the initial functional requirements. In System Analysis and Design, she developed the comprehensive **Use Case Diagram**, designed the **Sequence Diagrams** covering both system scenarios, and contributed to the database design phase during the Normalization process. For UI/UX and Presentation Design, she created the entire visual design, layout, and color scheme of the presentation, while curating and formatting all images, sizes, and structural design elements for the entire presentation. In Requirement Gathering and Documentation, she formulated and finalized the interview questions used for system requirements gathering and contributed to writing and structuring sections of the project Documentation.	33%
Shared Team Foundation		(The Core Project Idea) — This percentage represents the collective effort of the entire team. Before distributing individual tasks, all team members collaboratively participated in intensive brainstorming sessions to conceptualize the overall project theme, develop its core vision, and establish the main objectives of the system.	1%